

L E C T U R E 1 1

Spring Security

Tokens, JWT, OAuth 2.0, OpenID Connect

...and how to lock down the API we built last week.

Dejan Mitov · Ivana Rancic

UACS · CSCI 2450 · Computer Interface Programming · 2026

A confession

Right now, anyone can call your /friends API.

No login. No token. Just curl, and they're in.

```
$ curl http://your-server/friends
```

```
$ curl -X DELETE http://your-server/friends/1
```

Today we add three things

1. A login. So we know WHO is calling.
2. A token. So we don't ask for a password every request.
3. A rule. So only the right people can DELETE.

Learning outcomes

1

Tell apart

authentication from authorization, in plain words.

3

Decode

a JWT and read what's inside — by hand.

5

Distinguish

OAuth 2.0 from OpenID Connect — and when to use each.

2

Explain

why we moved from sessions to tokens.

4

Recognise

the four actors in OAuth 2.0 and what each does.

6

Secure

your Friends API with Spring Security.

Concept first. Code at the end.

1 Foundations

AuthN vs AuthZ.
What are we protecting?

2 Sessions

The old way. Why it struggles.

3 Tokens

Tickets, not passwords.

4 JWT

The standard token format.

5 OAuth 2.0

“Login with Google” explained.

6 OpenID

Adds identity to OAuth.

7 Spring

Lock down /friends.

PART 1 / 7

Foundations

Two words. Often confused. Always both.

Authentication vs Authorization

Authentication

"Who are you?"

You walk into the building.
The doorman checks your ID.

Examples: a password, a fingerprint, a token from your phone.

Authorization

"What can you do?"

You're now inside the building.
Which rooms have your name on them?

Examples: a role (ADMIN), a scope (read:friends), an ACL.

AuthN comes first. You can't authorize someone you haven't identified.

Anyone can do anything

Postman

GET /friends

Response

200 OK

```
[
  { "id": 1, "firstName": "Ada" },
  { "id": 2, "firstName": "Linus"}
]
```

⚠ Notice what's missing

- No “Authorization” header.
- No login asked for.
- No idea WHO sent the request.
- No idea WHAT they're allowed to do.
- Server cheerfully answers anyway.

Fixing this is the job for the next 50 minutes.

Every secure system, in any language

1

Identity

We know WHO is calling.

How

username + password,
fingerprint, token

2

Permission

We know WHAT they're allowed to
do.

How

roles, scopes, rules

3

Integrity

What they sent hasn't been
tampered with.

How

signatures, HTTPS, hashes

Spring Security gives us all three. We just have to point at the right pieces.

Three attackers, three angles

The eavesdropper

Listens on the wire.

Defence:

HTTPS / TLS encrypts everything.

The impersonator

Pretends to be someone else.

Defence:

Real authentication — passwords, tokens.

The tamperer

Changes the message in flight.

Defence:

Signatures and integrity checks.

PART 2 / 7

Sessions

The classic way: server remembers you between requests.

Username + password

1

User types

`alice / secret123`

2

Browser sends

`POST /login
{user, password}`

3

Server checks

`compare against
stored hash`

4

Welcome!

...now what?

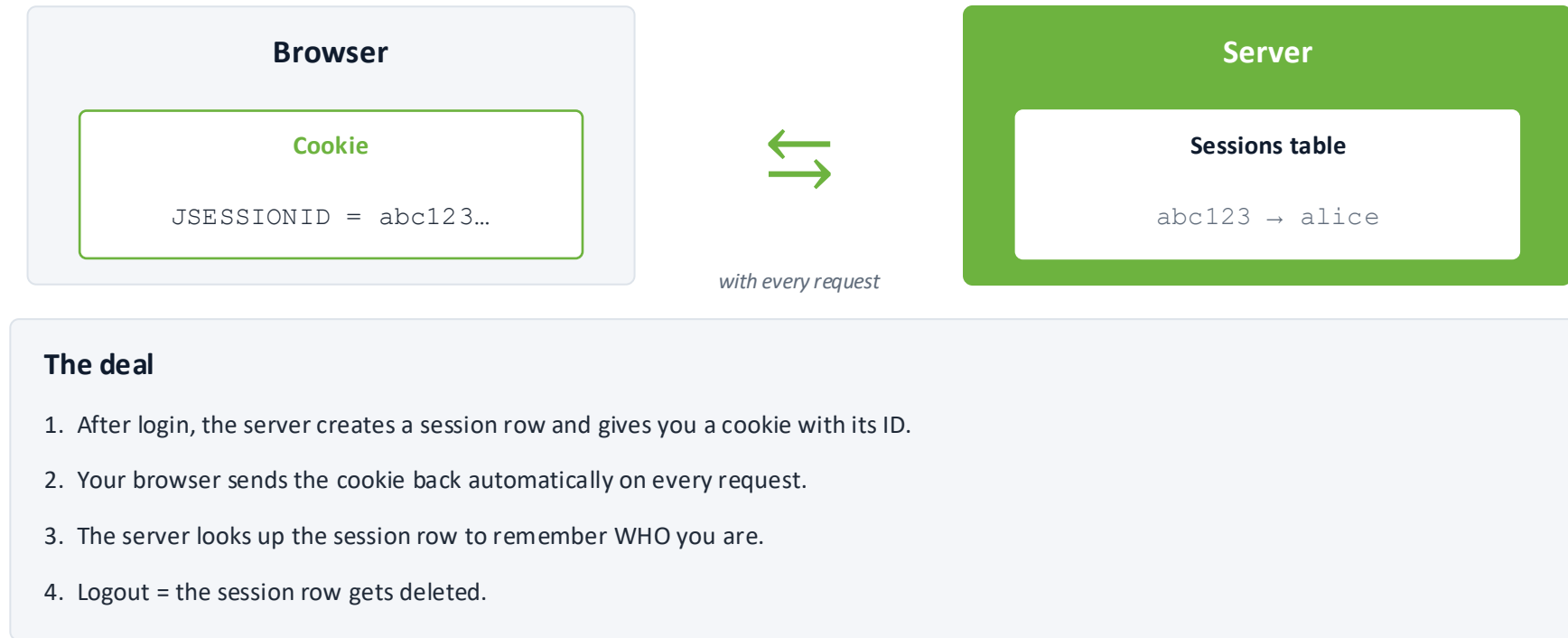
The real question: what about the NEXT request?

HTTP is stateless. The server forgets you the moment the response is sent.

Asking for the password every single request is unsafe and slow.

So we need to remember the user — somehow.

Server-side sessions



Three pains that show up at scale

1

Multiple servers

Server A made the session. Server B has no idea who you are.
Fix? Sticky load balancing or a shared session store. Both add complexity.

2

Mobile apps & SPAs

Cookies are a browser thing. Mobile apps and APIs across domains struggle with them — CORS, SameSite, the whole headache.

3

Many apps, many logins

Every site has its own session. Every site asks for its own password. Users end up reusing passwords. Bad.

Sessions are great for one-server, browser-only apps. For everything else — we want something better.

The Single Sign-On idea

Without SSO

- Each app has its own login form.
- Each app stores its own passwords.
- Each app handles its own breaches.
- Users reuse the same password everywhere.
- One leak puts every account at risk.

With SSO

- One trusted identity provider does the login.
- Apps trust the provider's verdict.
- Users have ONE password, in ONE place.
- “Login with Google” is exactly this.
- And we still need a way to PROVE it.

To make SSO work, the apps need a way to talk about you securely. Enter: tokens.

PART 3 / 7

Tokens

Replace passwords with something safer to pass around.

Tokens are tickets, not passwords

Sending the password every time

- A network sniffer sees it once → owned forever.
- Server has to verify it from the database every request.
- Hard to revoke for one device only.
- Hard to grant partial access (read-only, no delete).

Sending a token instead

- Issued ONCE after login, used many times.
- Has an expiry — leak today, useless tomorrow.
- Can carry permissions inside it.
- Can be signed so the server trusts it instantly.

Like a movie ticket: shows who, what film, what seat — and is invalid after the show.

By reference · By value

By Reference

"It's a coat-check ticket."

The token is a random string. Carries no info. The server keeps your details and looks them up by the string.

```
a8f3-9c1d-7e22-bd44
```

Server lookup every time. Easy to revoke.

By Value

"It's a boarding pass."

The token CARRIES your info — name, role, expiry. The server reads it and trusts the signature.

```
eyJhbGciOiJIUzI1Ni...
```

No DB lookup needed. Faster at scale.

By-value tokens are the modern default. The most famous format: JWT.

PART 4 / 7

JWT

JSON Web Token — the standard ticket of the web.

Three Base64 chunks, joined by dots

```
eyJhbGciOiJIUzI1NiJ9 . eyJzdWIiOiJhbGljZSI6ImV4cCI6MTcxNjI4OTAwMH0 . Sf2dxKL1z0wXp...
```

Header

Which algorithm signed this?

Payload

Who is the user, what can they do,
when does it expire?

Signature

Proof that nobody changed the header
or payload.

Defined in RFC 7519. Used by every cloud provider. Goes in the Authorization header.

Same JWT — decoded

Header

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

Payload

```
{  
  "sub": "alice",  
  "role": "USER",  
  "exp": 1716289000  
}
```

Signature

```
HMAC_SHA256(  
  base64url(header) + "." + base64url(payload),  
  serverSecret  
)  
  
// -> Sf2dxKL1z0wXp... (this is what proves nobody tampered)
```

Crucial misconception to clear up

 **Anyone can decode a Base64 string. No key, no password.**

Base64 just turns binary bytes into ASCII so they survive HTTP transport. It does not hide anything.

Why JWT uses Base64

- JSON has spaces and weird chars.
- Headers and URLs hate weird chars.
- Base64 makes it safe to ship.
- Adds about 33% size — worth it.

So... NEVER put in the payload

- ✗ Passwords (in any form).
- ✗ Credit card numbers.
- ✗ ID numbers, SSNs, secrets.
- ✗ Anything you wouldn't put on a postcard.

How the server seals the envelope

1

Header + Payload

Base64-encoded
and joined by '.'

2

Secret key

Known only
to the server

3

HMAC-SHA256

One-way maths
on the two above

4

Signature

Appended after
a third '.'

Why this works

- The signature depends on every byte of header + payload + the secret.
- Change one byte → the server's recomputed signature won't match → rejected.
- *Without the secret, you cannot fake a valid signature. That's the whole trick.*

How the server checks the token

1

Receive

Client sends header:
Authorization: Bearer <jwt>

2

Split

Pull out header.payload.signature.

3

Re-compute

Same HMAC, same secret, same input.

4

Compare

Match? Trust it.
Different? 401.

Notice: no database call. The token verifies itself. That's why JWT scales.

Same /friends call, with a token



Postman

GET /friends

Headers

Authorization:
Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWIi...

Response

200 OK

```
[
  { "id": 1, "firstName": "Ada" }
]
```

What happened

1. Client put the JWT in the Authorization header.
2. Server checked the signature with its secret.
3. Server read the payload — it knows it's “alice”.
4. Server checked exp — token still valid.
5. Server answered the request.
6. Zero database calls for authentication.

Same security as a session — without the server-side state.

PART 5 / 7

OAuth 2.0

“Login with Google” — explained, not memorized.

Imagine this conversation

App	<i>"Hi! I'd like to read your Gmail contacts."</i>
User	<i>"OK, but I'm not giving you my Gmail password."</i>
App	<i>"Fair. Let me ask Google directly?"</i>
User	<i>"Yes — but only contacts. Not my emails."</i>
Google	<i>"Sure. Here's a temporary token. Contacts only."</i>

OAuth 2.0 = the rulebook for that whole conversation, in HTTP.

Everyone in OAuth has one of four roles

1 Resource Owner

The user. Owns the data.

e.g. You

2 Client

The app that wants the data.

e.g. Crypto Portfolio app

3 Authorization Server

Issues tokens after the user agrees.

e.g. Google · Facebook

4 Resource Server

Holds the data. Trusts the token.

e.g. Gmail API

Get these four straight, and OAuth makes sense forever.

Same four roles. Familiar setting.

1

Passenger

≈ Resource Owner

Wants to travel. Owns the journey.

2

Travel agent

≈ Client

Buys the ticket on the passenger's behalf.

3

Ticket machine

≈ Authorization Server

Validates and prints the ticket.

4

Train station

≈ Resource Server

Lets you in only with a valid ticket.

Tokens are tickets. You hand them over, you get through the gate. No need to show your passport every time.

The standard, secure OAuth flow

- 1 Redirect** App sends user to Google's login page.
- 2 Login** User logs in to Google directly. App never sees the password.
- 3 Consent** Google asks: "App X wants to read your contacts. OK?"
- 4 Code** Google redirects user back with a short-lived authorization code.
- 5 Exchange** App sends the code + its own client secret back to Google. Gets a token.
- 6 Use** App calls Gmail API with `Authorization: Bearer <token>`.

Why two steps (code → token)? Codes can leak; without the client secret they're useless.

OAuth, told as a story

The setup

Victoria uses Crypto Portfolio (a third-party app) to track her holdings.

She wants to log in with Facebook — without giving Crypto Portfolio her Facebook password.

1. Click "Log in with Facebook"

Crypto Portfolio redirects Victoria's browser to Facebook.

2. Victoria logs in to Facebook

Facebook is the only one that ever sees her password.

3. Facebook asks: "Allow Crypto Portfolio to know who you are?"

She clicks Approve.

4. Facebook redirects back with a code

...which Crypto Portfolio swaps for an access token.

5. Crypto Portfolio uses that token

Now it knows it's Victoria, and acts on her behalf — without her password.

OAuth 2.0 was designed for AUTHORIZATION

OAuth 2.0 answers "What can this app do on my behalf?"

It does NOT, on its own, answer "WHO is using the app right now?"

What developers did

They used the access token as proof of identity:
"You have a token, so I'll trust it's Victoria."

Risky: tokens are about permissions, not identity.

What we should do

Use a layer that explicitly identifies the user — designed to sit on top of OAuth.

Enter: OpenID Connect.

PART 6 / 7

OpenID Connect

Adds identity to OAuth — same flow, one extra ticket.

OIDC = OAUTH + IDENTITY

Same flow. One extra token.

OAuth 2.0 alone

Returns: 1 token

```
access_token
```

Means: "this app may read your contacts."

Says nothing about WHO the user is.

OpenID Connect

Returns: 2 tokens

```
access_token
```

```
id_token ← new
```

The id_token IS a JWT. Its payload tells you exactly who logged in.

OAuth handles permission. OIDC adds identity. Most modern logins are OIDC.

It's a JWT — you already know how to read it

```
// id_token payload — decoded
{
  "iss": "https://accounts.google.com", // who issued it
  "sub": "110169484474386276334",      // stable user id
  "aud": "crypto-portfolio",           // who it's for
  "name": "Victoria Lee",
  "email": "victoria@example.com",
  "iat": 1716285400,                   // issued at
  "exp": 1716289000                    // expires at
}
```

Read this once on the client → you know who logged in. That's it.

The access_token is still used to call APIs. The id_token is consumed by your app.

JWT, OAuth 2.0, OpenID Connect

JWT

The messenger.

Carries data.
Three base64 chunks. Signed.

OAuth 2.0

The authorization engine.

Hands out tokens.
Decides permissions.

OpenID Connect

The authentication engine.

Sits on top of OAuth.
Adds an ID token.

If you remember nothing else: OIDC sits on OAuth, and the ID token is just a JWT.

PART 7 / 7

Spring Security

Now lock down the Friends API we built last week.

Why your project uses a simpler auth

1

Authentication is its own course.

Sessions, JWT signatures, OAuth flows — every line of it has edge cases. Today's lecture introduces the concepts so you recognise them at work.

2

For your project, we hold authentication constant.

A single `X-API-Key` header, checked once in a `HandlerInterceptor`. No login. No tokens. No refresh. No sessions.

3

Why?

So your effort goes into the layers we're actually grading — REST design, validation, the business invariant, error handling. The auth layer is a placeholder you'll replace in your first job.

Same place in the request flow. Easier mechanism.

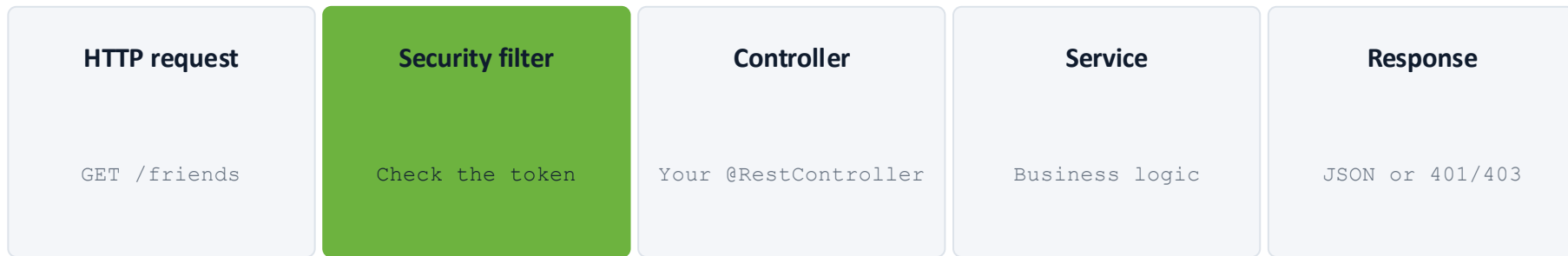
In this lecture: **Authorization: Basic ...** In your project: **X-API-Key: demo-key-2026**

Who can call which endpoint?

Endpoint	Public?	Logged-in user	Admin only
POST /auth/login	✓	—	—
GET /friends	—	✓	✓
POST /friends	—	✓	✓
PUT /friends/{id}	—	✓	✓
DELETE /friends/{id}	—	—	✓

Three rules to express in code: public · authenticated · ROLE_ADMIN.

Before your controller ever runs



The security filter runs FIRST.

- No token, or invalid token → 401 Unauthorized. Controller never runs.
- Token valid, but role wrong → 403 Forbidden. Controller never runs.
- All good → request continues to your `@RestController`, which now KNOWS who's calling.

One line in pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Restart and watch what changes

- Every endpoint is locked behind HTTP Basic auth — even /friends.
- A random password is printed to the console at start-up.
- The default user is `user`. Use that + the random password to call your API.

Now — let's replace those defaults with our own rules.

Never plain text. Always BCrypt.

Rule: never store the password the user typed. Store its HASH.

BCrypt is one-way: easy to compute, infeasible to reverse, slow on purpose to slow attackers.

```
@Bean
PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}

// On registration:
user.setPassword(passwordEncoder.encode(rawPassword));

// On login (Spring Security does this for us):
passwordEncoder.matches(rawPassword, user.getPassword());
```

A stored password looks like: \$2a\$10\$N9qo8uLOickgx2ZMRZoMye... Useless without the secret.

Tell Spring our three rules

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain api(HttpSecurity http) throws Exception {
        return http
            .csrf(csrf -> csrf.disable()) // stateless API
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/auth/**").permitAll() // public
                .requestMatchers(HttpMethod.DELETE, "/friends/**")
                    .hasRole("ADMIN") // admins only
                .anyRequest().authenticated() // everyone else logged in
            ).httpBasic(Customizer.withDefaults()) // for class - JWT later
            .build();
    }
}
```

Two new files. Spring will find them automatically.

AppUser.java — the entity

```
@Entity
public class AppUser {
    @Id @GeneratedValue Long id;

    @Column(unique = true)
    private String username;

    private String password; // BCrypt hash

    private String role;      // USER or ADMIN

    // + getters/setters
}
```

UserService — Spring asks for users

```
@Service
public class UserService implements
    UserDetailsService {

    @Override
    public UserDetails loadUserByUsername(String
u) {
        AppUser a = repo.findByUsername(u)
            .orElseThrow();
        return User
            .withUsername(a.getUsername())
            .password(a.getPassword())
            .roles(a.getRole())
            .build();
    }
}
```

What anonymous calls now get back

Postman

GET /friends

Response

401 Unauthorized

```
{
  "error": "Unauthorized",
  "status": 401
}
```

Postman

DELETE /friends/1

Headers

(logged in as USER, not ADMIN)

Response

403 Forbidden

```
{
  "error": "Forbidden",
  "status": 403
}
```

401 = "who are you?" · 403 = "I know you, but no."

Same calls, this time logged in

Postman

GET /friends

Headers

Authorization:
Basic YWxpY2U6c2VjcmV0

Response

200 OK

```
[  
  { "id": 1, "firstName": "Ada" }  
]
```

Postman

DELETE /friends/1

Headers

Authorization:
Basic YWRtaW46YWRtaW4xMjM=

Response

200 OK

(empty body – friend deleted)

Same code as last week. Same endpoints. Now with identity and rules.

When you're ready

1

Stage 1 — HTTP Basic

Username + password sent on every call. Simple to start.

✓ Good for class, quick prototypes.

✗ Credentials on the wire constantly. Don't ship like this.

2

Stage 2 — Login + JWT

Login once → receive a JWT → send it as Bearer on every call.

✓ Stateless. Mobile-friendly. Standard for APIs.

✗ You manage signing keys and expiries.

3

Stage 3 — OIDC provider

Outsource login to Google / GitHub / Keycloak. Receive an ID token.

✓ No password storage. SSO across apps.

✗ More moving parts to wire up.

Your final project is fine at Stage 1 or 2. Stage 3 is for after this course.

Top to bottom, in plain words

1

Foundations

AuthN ≠ AuthZ. Identity, permission, integrity.

2

Sessions

Server-side state. Works, doesn't scale.

3

Tokens

Tickets, not passwords. By-reference vs by-value.

4

JWT

header.payload.signature. Base64 ≠ encryption.

5

OAuth 2.0

Four actors. Railway analogy. Authorization code grant.

6

OpenID Connect

OAuth + identity. ID token = JWT.

7

Spring Security

Filter chain in front. Three rules. /friends locked.

Six things to remember

1 Two questions

Authentication asks WHO. Authorization asks WHAT.

3 JWT = signed JSON

Three Base64 chunks. Server trusts the signature, not the user.

5 OIDC = identity on top

OAuth + ID token (a JWT). "Login with Google" is OIDC.

2 Tokens beat passwords

A short-lived ticket is safer than the password itself.

4 OAuth = permissions

Four actors. The user never gives the app the password.

6 Spring runs the filter

Block unauthenticated traffic before your code ever runs.

Don't fall for them

✗ Lie

"We hash with MD5, that's safe enough."

✓ Truth

MD5 was broken in the 90s. Modern GPUs crack it in minutes. Use BCrypt or Argon2.

✗ Lie

"Our token is Base64-encoded so it's encrypted."

✓ Truth

Base64 is reversible by anyone, with no key. It's encoding, not encryption. Sign your JWTs and use HTTPS.

Three sentences that will travel with you forever

JWT

I am the messenger. I carry signed JSON between client and server.

OAuth 2.0

I am the rule-maker. I decide which apps can do what on the user's behalf.

OpenID Connect

I am the identity layer. I tell apps who the user actually is.

Together: JWT carries the message. OAuth grants the access. OIDC names the user.

END OF LECTURE 11

Questions?

HOMework

Lock down your Friends API

Add Spring Security, BCrypt, two roles. /friends authenticated, DELETE admin-only.

NEXT WEEK

Lecture 12 — Architecture, Testing, Deployment

From your laptop to a real server. Docker, CI/CD, the whole pipeline.